Documentation technique

Documentation technique..... 1
Introduction.....2
1. Conception de l’architecture.....2
2. Présentation de l’architecture.....3
2.1. Le package common.....3
2.2. Le package mvc..... 4
2.3. Le package mvc.entity.....5
2.6. Le package mvc.game..... 8
2.7. Le package mvc.menu..... 9
2.8. Le package mvc.leveleditor.....10
2.9. Le package application.....10
3. Fonctionnalités du jeu.....11
3.1. Lancement du jeu..... 11
3.2. Gestion du héros..... 11
3.3. Déplacements et exploration..... 11
3.4. Gestion des objets.....12
3.5. Système de combat.....12
3.6. Génération procédurale.....13
3.7. Interface graphique..... 13
3.8. Interface CLI.....13
3.9. Changement de langue.....13
3.10. Éditeur de niveau..... 14
4. Reprise du projet et évolutivité.....14
5. Conclusion.....15

Introduction

Ce document technique a pour objectif de décrire l'architecture logicielle du projet et permettre à de futurs développeurs de reprendre plus facilement le travail.
Il présente l'organisation générale du code, le rôle des principaux packages, ainsi que les grandes fonctionnalités actuellement prises en charge par l'application.

1. Conception de l'architecture

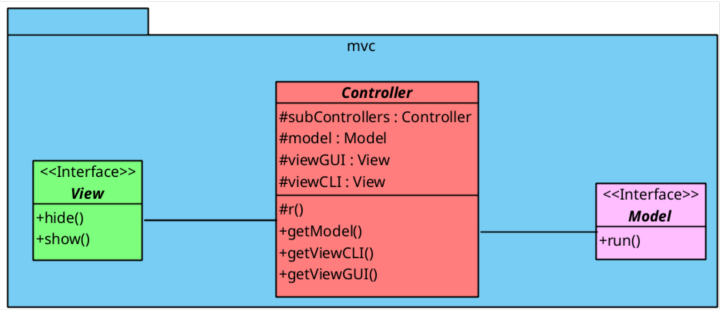
L'un des objectifs majeurs de ce projet n'était pas uniquement d'ajouter une interface graphique, mais également d'adapter le jeu à une architecture MVC, conformément au format demandé dans le cadre du cours.

Avant même l'implémentation des nouvelles fonctionnalités, il a donc fallu repenser l'organisation du projet afin de respecter cette contrainte. Nous avons choisi de nous appuyer sur le MVC fourni par le professeur, qui impose une séparation stricte entre les composants : le modèle et la vue ne communiquent jamais directement ; seul le contrôleur assure la liaison entre eux.

Dans cette architecture :

- le Model contient les données et les traitements
- la View gère l'affichage
- le Controller fait le lien entre les deux et centralise les interactions

Ce choix nous a permis de séparer clairement la logique métier, l'affichage et la gestion des actions utilisateur, tout en gardant une structure suffisamment souple pour supporter à la fois une interface graphique et une interface en ligne de commande.



Cependant, appliquer cette architecture à l'ensemble du projet dans trois packages uniques model, view et controller aurait rapidement rendu le code difficile à lire et à maintenir. Nous avons donc choisi une organisation plus modulaire, en structurant d'abord le projet par domaines fonctionnels, puis en appliquant à l'intérieur de chacun une organisation de type MVC.

Nous retrouvons ainsi plusieurs grands ensembles :

- common
- mvc
- application

Le package common contient le cœur métier du jeu, c'est-à-dire les classes indépendantes de l'interface.
Le package mvc contient l'architecture applicative, organisée par domaines puis par couches MVC.
Enfin, le package application regroupe les points d'entrée et les classes de lancement.

Cette structure a rendu le projet plus clair, plus modulaire et plus facile à faire évoluer.

2. Présentation de l'architecture

2.1. Le package common

Le package common regroupe les classes représentant les éléments fondamentaux du jeu.
Il s'agit du cœur logique de l'application, indépendant de toute interface utilisateur.

On y retrouve notamment :

- les entités du jeu (Hero, Enemy, Archer, Berserker, etc.)
- les objets (Item, Weapon, Scroll, Chest, Key, etc.)
- la gestion des salles et des sorties (Room, Exit, LockedExit, SimpleExit)
- la génération procédurale du donjon (DungeonGenerator, DungeonData)
- la gestion de la langue (Languages)
- les structures de données de l'éditeur de niveau (LevelData, RoomEditorData, LevelRegistry, LevelDataConverter).

Ce package contient donc la logique métier brute, sans dépendance directe vers les vues.

2.2. Le package mvc

Le package mvc constitue l'ossature principale de l'application.
Il repose sur la base fournie par le professeur, à savoir les interfaces et classes génériques Model, View et Controller.

À partir de cette base, nous avons organisé le projet en plusieurs sous-packages spécialisés :

- entity
- map
- item
- game
- menu
- leveleditor

Chacun de ces domaines est lui-même structuré autour de sous-packages :

- model
- controller
- view/base
- view/gui
- view/cli

Cette organisation permet d'avoir un MVC local pour chaque grande partie du projet, ce qui améliore fortement la lisibilité.

2.3. Le package mvc.entity

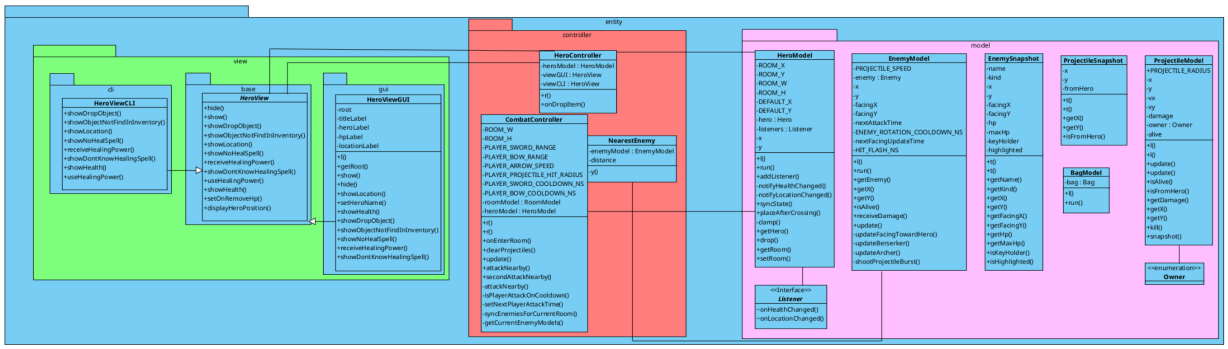
Le package mvc.entity est dédié à la gestion des personnages du jeu, en particulier le héros, les ennemis et le système de combat.

Il contient notamment :

- les modèles représentant l'état des héros, ennemis et projectiles ;
- les contrôleurs liés au héros et aux combats ;
- les vues associées au héros, en version CLI et GUI.

Cette partie gère donc :

- l'état du héros ;
- les points de vie ;
- les dégâts ;
- les ennemis ;
- les projectiles ;
- le système de combat.



2.4. Le package mvc.map

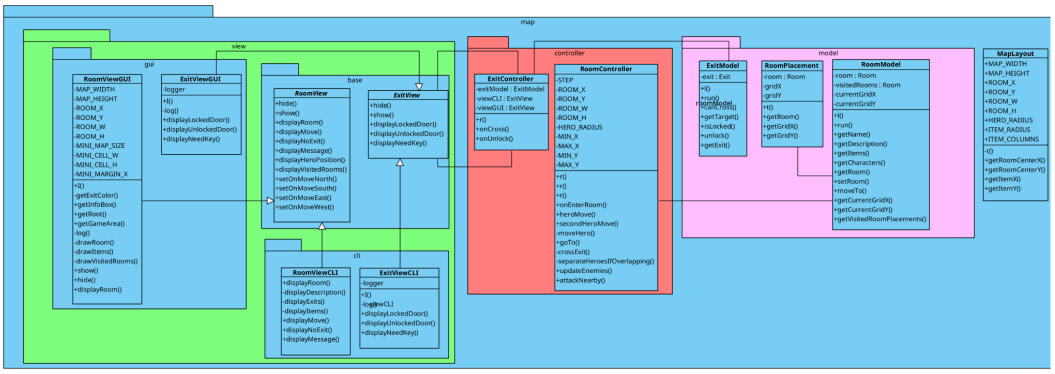
Le package mvc.map regroupe tout ce qui concerne la navigation dans les salles et la représentation spatiale du jeu.

Il contient :

- les modèles des salles et des sorties ;
- les contrôleurs de déplacement et de franchissement d'exits ;
- les vues de salle en CLI et GUI ;
- les outils liés à la minimap et au placement dans l'espace.

Cette partie gère notamment :

- les déplacements du héros dans la salle ;
- le passage d'une salle à une autre ;
- les sorties verrouillées ;
- l'affichage de la salle ;
- la minimap ;
- les salles déjà visitées.



2.5. Le package mvc.item

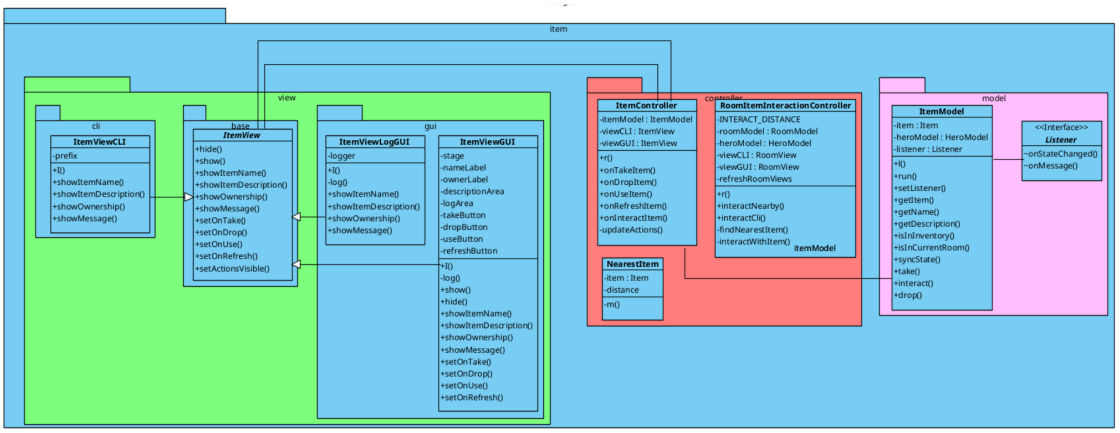
Le package mvc.item s'occupe de la gestion des objets et des interactions avec ces derniers.

Il contient :

- les modèles d'objets ;
- les contrôleurs d'interaction avec les objets ;
- les vues associées, aussi bien en interface graphique qu'en interface textuelle.

Cette partie permet de gérer :

- le ramassage d'objets ;
- le dépôt d'objets ;
- l'utilisation d'objets ;
- l'ouverture des coffres ;
- les interactions de proximité avec les objets présents dans une salle.



2.6. Le package mvc.game

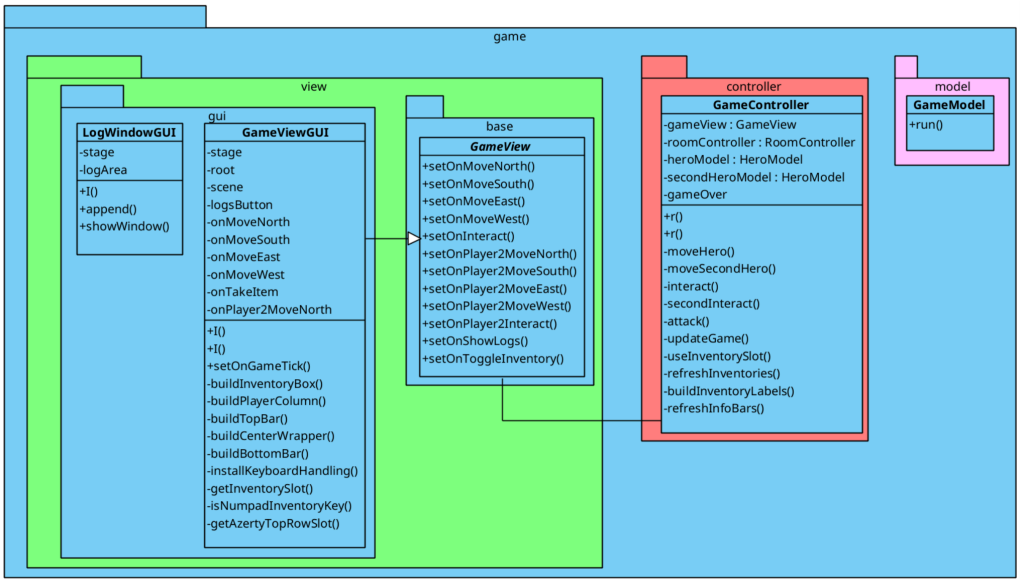
Le package mvc.game correspond à la coordination générale de la partie en cours.

Il contient :

- le modèle global de jeu ;
- le contrôleur principal du déroulement d'une partie ;
- la vue principale de jeu ;
- la fenêtre de logs.

Cette partie centralise notamment :

- la boucle de jeu ;
- les déplacements ;
- les attaques ;
- l'inventaire ;
- l'affichage des informations de partie ;
- le menu pause ;
- le game over ;
- l'affichage global de l'interface.



2.7. Le package mvc.menu

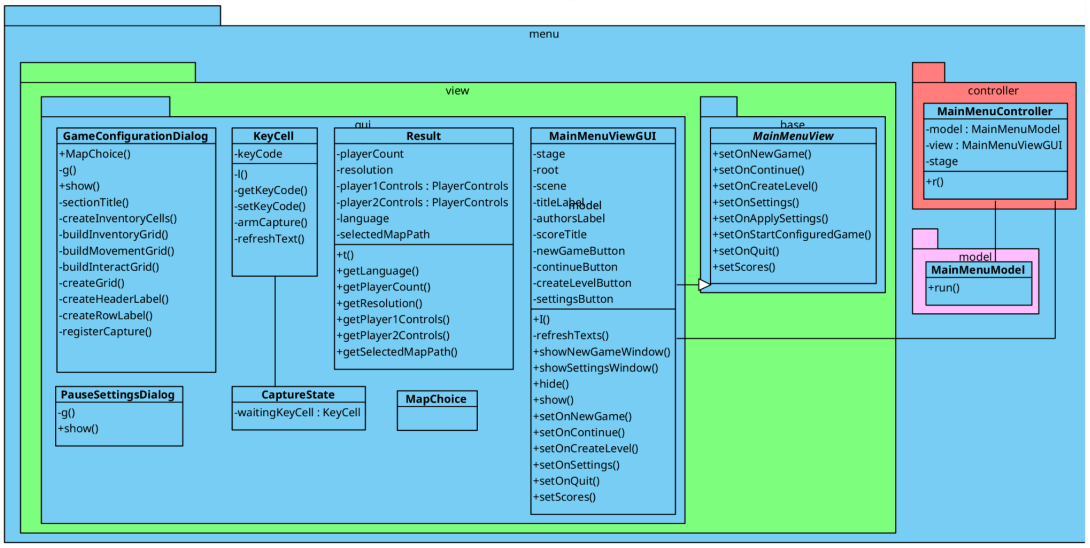
Le package mvc.menu regroupe tout ce qui concerne le menu principal et les fenêtres de configuration.

Il permet :

- d'ouvrir le menu principal ;
- de lancer une nouvelle partie ;
- de configurer le nombre de joueurs ;
- de modifier les contrôles ;
- de changer la langue ;
- de modifier la résolution ;
- d'accéder à l'éditeur de niveau.

Cette partie contient notamment :

- le modèle du menu ;
- le contrôleur du menu principal ;
- la vue du menu ;
- les boîtes de dialogue de configuration.



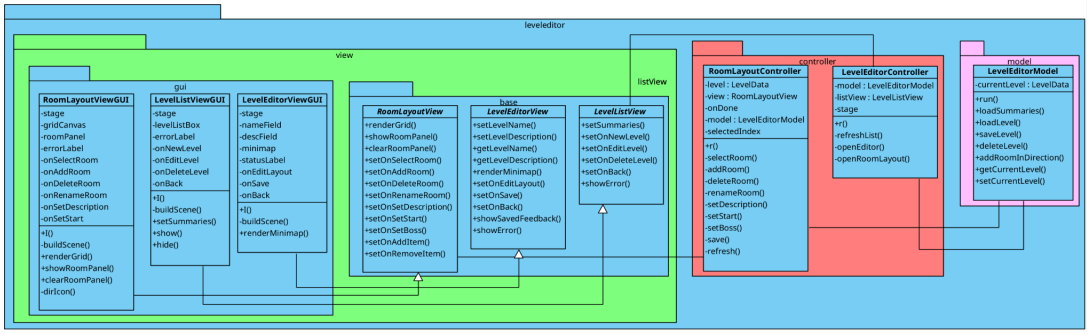
2.8. Le package mvc.leveleditor

Le package mvc.leveleditor contient l'ensemble du code lié à l'éditeur de niveau.

Il permet :

- de créer un nouveau niveau ;
- de charger un niveau existant ;
- de modifier l'agencement des salles ;
- de définir une salle de départ ;
- de définir une salle de boss ;
- d'ajouter ou supprimer des objets ;
- d'ajouter ou supprimer des ennemis ;
- d'enregistrer les niveaux personnalisés.

Cette partie constitue une fonctionnalité importante du projet, puisqu'elle permet d'étendre le jeu avec des niveaux créés manuellement.



2.9. Le package application

Le package application regroupe les classes de lancement de l'application.

On y retrouve par exemple :

- le point d'entrée principal du programme ;
- le lancement du menu principal ;
- le lancement d'une partie ;
- le lancement de l'éditeur de niveau.

Ce package ne contient pas la logique métier du jeu, mais orchestre le démarrage général de l'application.

3. Fonctionnalités du jeu

Le projet propose actuellement plusieurs fonctionnalités principales.

3.1. Lancement du jeu

Le jeu peut être lancé de différentes manières :

- via une interface graphique ;
- via une version ligne de commande ;
- via un lancement direct d'une partie aléatoire ;
- via le menu principal.

3.2. Gestion du héros

Le joueur contrôle un héros disposant :

- de points de vie ;
- d'un inventaire
- d'armes
- d'une position dans la salle
- d'une position dans le donjon.

Le projet gère également un mode deux joueurs, avec un second héros et des contrôles séparés.

3.3. Déplacements et exploration

Le joueur peut se déplacer librement dans une salle, puis traverser les sorties pour progresser dans le donjon.

Le système gère :

- les sorties simples
- les sorties verrouillées
- les transitions entre salles
- les salles visitées
- l'exploration progressive
- la minimap

3.4. Gestion des objets

Le jeu propose plusieurs catégories d'objets :

- armes
- parchemins
- clés
- coffres
- objets simples

Le joueur peut :

- ramasser un objet
- l'utiliser
- l'équiper
- le déposer
- ouvrir un coffre

3.5. Système de combat

Le système de combat prend en compte :

- les attaques du héros
- les armes de mêlée
- les armes à distance
- les projectiles
- les ennemis à comportements différents
- les dégâts
- les récompenses après élimination

Certains ennemis attaquent au corps à corps, d'autres à distance, ce qui enrichit le gameplay.

3.6. Génération procédurale

Le jeu peut générer automatiquement un donjon contenant :

- une salle de départ
- plusieurs salles reliées entre elles
- une salle de boss
- des ennemis
- des objets
- une clé permettant d'ouvrir certaines zones

Cette génération automatique permet de varier les parties.

3.7. Interface graphique

La version graphique du jeu permet :

- l'affichage du héros et des ennemis
- les déplacements en temps réel
- l'affichage de l'inventaire
- les logs
- le menu pause
- le game over
- le changement de résolution
- la gestion de plusieurs joueurs

3.8. Interface CLI

Le projet dispose aussi d'une interface en ligne de commande pour certaines parties du jeu.

Cela montre que l'architecture adoptée permet bien d'utiliser plusieurs types de vues sur une même logique applicative.

3.9. Changement de langue

Le projet intègre un système de traduction centralisé via la classe Languages.

Cela permet d'afficher les textes du jeu dans plusieurs langues selon la configuration choisie.

3.10. Éditeur de niveau

L'éditeur de niveau permet de créer et modifier des niveaux personnalisés.

L'utilisateur peut :

- créer un niveau
- modifier son nom et sa description
- organiser les salles
- gérer les connexions
- définir les salles spéciales
- ajouter objets et ennemis
- sauvegarder ou charger ses créations

Ces niveaux peuvent ensuite être convertis en niveau jouable dans le jeu.

4. Reprise du projet et évolutivité

L'architecture retenue a été pensée pour faciliter la reprise du code.

Pour ajouter une nouvelle fonctionnalité, il est généralement possible de suivre le raisonnement suivant :

1. identifier le domaine concerné (entity, item, map, etc.)
2. modifier ou créer la classe métier dans common
3. adapter le modèle correspondant
4. adapter le contrôleur
5. modifier ou compléter les vues concernées

Cette structure permet d'éviter qu'une modification locale ait des conséquences trop importantes sur tout le projet.

Il reste cependant indispensable de respecter la règle principale du MVC imposé : le modèle et la vue ne doivent jamais communiquer directement. Toute interaction doit passer par le contrôleur.

5. Conclusion

Le projet repose sur une architecture inspirée du MVC fourni par le professeur, adaptée à la structure de notre jeu.

Nous avons choisi de combiner :

- une séparation stricte entre modèles, vues et contrôleurs
- une organisation par domaines fonctionnels
- un noyau métier indépendant de l'interface

Cette architecture nous a permis de conserver un projet lisible, modulaire et évolutif, tout en intégrant à la fois :

- une interface graphique
- une interface CLI
- un système de combat
- une génération procédurale
- un éditeur de niveau
- un système multilingue

Elle constitue donc une base solide pour poursuivre le développement du projet ou en faciliter la reprise ultérieure.